LANGCHAIN 마이그레이션 도우미 · 팀 모두를 위한 안내서

프로젝트 한눈에

최신 버전 v2.1 · 2026-10-13 · 새 설계, 새 역할, 팀 저장소 기준 반영

우리 팀이 무엇을, 왜, 어떤 순서로 만드는지 **주방 이야기**와 그림으로 설명합니다.

읽는 시간 약 10분 · 다음 문서: 02 화면·평가 파트 (4번 담당) · 03 팀 진행 상황

이것 세 줄만 기억해도 됩니다

1. AI는 **옛날 주방**만 기억해서, 도구가 이사 간 줄 모르고 옛날 레시피를 줍니다.
2. 우리 프로그램은 **새 주방 안내서**를 보고 레시피를 고치고, **서랍을 직접 열어** 도구와 그 기능이 있는지 확인합니다. 틀리면 다시 고칩니다.
3. 넷이 나눠 맡습니다: **연결**(1번) · **안내서**(2번) · **요리사와 검수원**(3번) · **손님 테이블과 심사위원**(4번).

1장

문제: 도구가 이사 갔는데 AI는 모릅니다

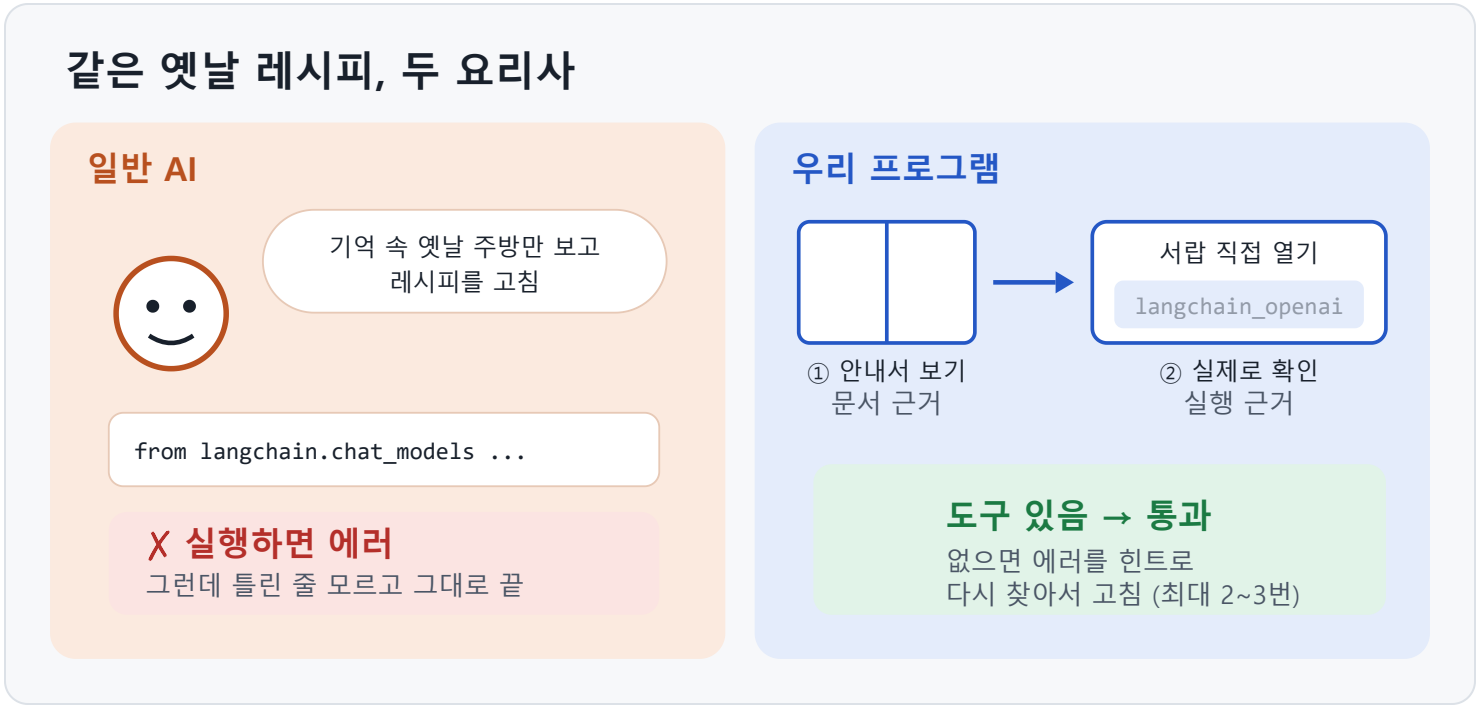
LangChain은 AI 프로그램을 만드는 주방 도구 세트이고, 코드 맨 위의 `import`는 "어느 서랍에서 어떤 도구를 꺼내라"는 뜻입니다. 2025년 1.0에서 주방이 새로 정리됐습니다.



옛날 서랍을 열면 실제로 이렇게 됩니다 (langchain 1.4.4에서 실행).

```
from langchain.chat_models import ChatOpenAI
ImportError: cannot import name 'ChatOpenAI'   ← "그 칸에 그런 도구 없어요"
```

인터넷에는 옛날 레시피가 훨씬 많아서, AI는 옛날 위치를 알려 주고 **틀린 줄도 모릅니다.** 이걸 "버전 지체 환각"이라고 부릅니다.



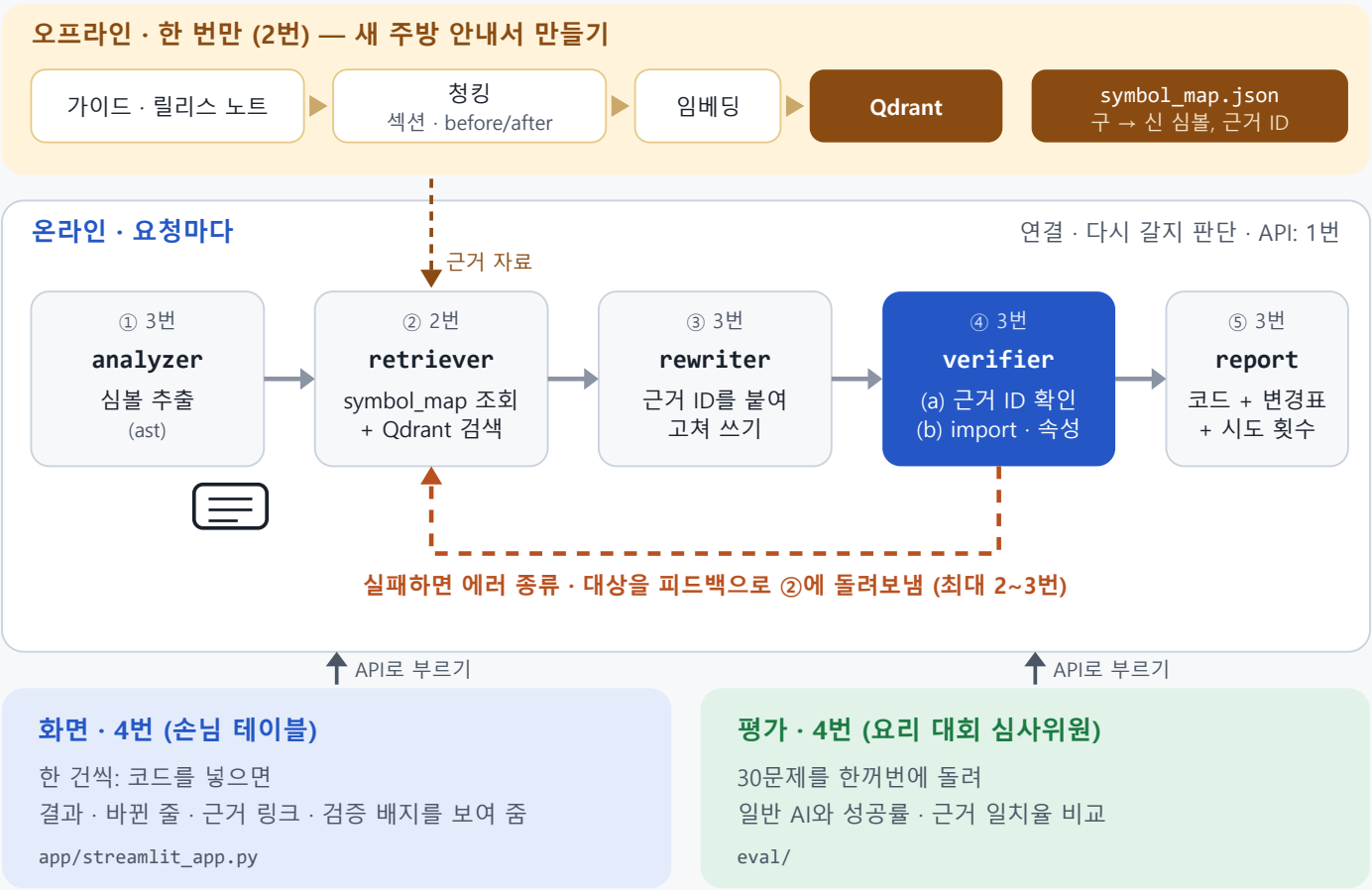
우리 프로그램은 두 번 확인합니다. 안내서를 보고 고치고(**문서 근거**), 서랍을 직접 열어 봅니다(**실행 근거**). 그래서 **"이중 검증"**입니다.

2장

기본 설계: 앞에서부터 보기

프로그램은 두 부분으로 나뉩니다. **한 번만 하는 준비**(안내서 만들기)와 **요청이 올 때마다 하는 일**(레시피 고치기)입니다. 레시피 카드가 움직이는 걸 따라가 보세요.

새 설계 한눈에 보기



단계	주방 말로	담당
오프라인 준비	가이드와 릴리스 노트로 **새 주방 안내서**(Qdrant)와 **이사 목록**(symbol_map: 옛 위치 → 새 위치)을 한 번 만들어 둠	2번
① analyzer	레시피에서 어떤 도구를 꺼내는지 목록 뽑기	3번

② retriever	이사 목록과 안내서에서 새 위치 찾기	2번
③ rewriter	안내서 쪽수(근거 ID)를 적어 가며 레시피 고쳐 쓰기	3번
④ verifier	(a) 적은 안내서 쪽수가 진짜 있는지 (b) 서랍을 열어 도구와 그 기능이 있는지 확인	3번
⑤ report	고친 레시피 + 바뀐 목록 + 몇 번 만에 됐는지 (영수증)	3번 (4번 가능, 확인 중)

되돌아가는 길 (검증 루프)

④에서 막히면 "무엇이 왜 없었는지"를 적어 ②로 돌려보내고 다시 찾습니다. 최대 2~3번. 일반 AI에는 이 길이 없어서 틀려도 그냥 끝납니다. 다시 갈지 판단하고 단계들을 잇는 일은 1번이 맡습니다.

헷갈리기 쉬운 점

우리 프로그램은 **주방을 정리하지 않습니다.** 주방은 LangChain 회사가 이미 정리했습니다. 우리가 고치는 것은 **사용자의 레시피(코드)**입니다.

3장

누가 무엇을 맡나

번호	역할	맡는 것	주방 비유
1번	그래프 · 백엔드	단계 연결, 다시 갈지 판단, `run_migration()` 함수, 샌드박스 환경	주방 동선과 주문 전달
2번	데이터 · 검색	안내서 만들기(Qdrant, symbol_map), retriever	안내서 만들기와 찾아주기
3번	변환 · 검증	analyzer, rewriter, verifier, report	요리사와 검수원
4번	화면 · 평가	Streamlit 화면, 평가 실험 + 보고서 · README · 발표	손님 테이블과 심사위원

1번과 3번의 차이

1번은 **직원들을 어떤 순서로 세우고 연결하나**, 3번은 **각 직원이 실제로 무슨 일을 하나**입니다. verifier는 통과/실패만 적고, 다시 갈지는 1번의 연결이 정합니다. 자세한 경계는 역할분담표에 있습니다.

4장

용어 사전

LangChain	AI 프로그램을 만드는 주방 도구 세트
import	"어느 서랍에서 어떤 도구를 꺼내라"는 코드 한 줄
속성	도구에 달린 기능. 예: `PromptTemplate.from_template`의 `from_template`
마이그레이션	옛날 레시피를 새 주방에 맞게 고치는 일
버전 지체 환각	AI가 옛날 주방만 기억해서 틀린 위치를 자신 있게 알려 주는 현상
RAG	AI가 답하기 전에 안내서(문서)를 찾아보게 하는 방법
Qdrant	안내서 조각을 넣어 두고 비슷한 내용을 빨리 찾는 책장 (벡터 DB)
symbol_map	이사 목록. 옛 위치 → 새 위치와 그 근거
근거 ID	"안내서 몇 쪽을 보고 고쳤는지" 적어 둔 번호
샌드박스	시험용으로 따로 만든 깨끗한 새 주방. 여기서 서랍을 열어 봄
검증 루프	막히면 이유를 적어 다시 찾고 고치는 되돌아가는 길 (최대 2~3번)
State	직원들이 같이 보는 주문서. 각 단계가 칸을 채움
API	화면과 평가가 프로그램을 부르는 주문 창구. 팀 기본안은 함수 `run_migration(code, target_version)` 하나
목표 버전	어느 버전으로 고칠지 고르는 값 (`target_version`, 지금은 1.x)

5장

확인 퀴즈

답을 누르면 바로 알려 줍니다. **0 / 5 맞힘**

Q1. from langchain.chat_models import ChatOpenAI 를 최신 LangChain에서 실행하면?

잘 실행된다

에러가 난다. ChatOpenAI가 langchain_openai 서랍으로 이사 갔기 때문

경고만 나온다

> 해설: 1.0에서 OpenAI 관련 도구는 langchain_openai라는 새 서랍장으로 이사 갔습니다.

Q2. Qdrant와 symbol_map은 주방으로 치면?

요리사

새 주방 안내서와 이사 목록

손님 테이블

> 해설: 2번이 한 번 만들어 두는 근거 자료입니다. retriever가 여기서 새 위치를 찾습니다.

Q3. verifier가 확인하는 두 가지는?

(a) 적은 근거가 진짜 있는지 (b) 서랍에 도구와 기능이 있는지

속도와 가격

글씨체와 색깔

> 해설: (a)는 문서 근거를, (b)는 실행 근거를 확인합니다.

Q4. verifier에서 실패하면 다시 갈지는 누가 정하나요?

- verifier 자신 (3번)
- 단계 연결 (1번)
- 화면 (4번)

> 해설: verifier는 결과와 이유만 적고, 다시 갈지는 1번이 만든 연결(조건부 엣지)이 정합니다.

Q5. 4번(화면 · 평가)이 맡는 것은?

- 안내서 만들기
- 레시피 고쳐 쓰기
- 손님 테이블(화면)과 심사위원(평가)

> 해설: 결과를 보여 주는 화면과, 30문제로 효과를 증명하는 평가를 맡습니다.

최신 버전 v2.1 · 2026-10-13 · 같은 내용의 MD와 PDF가 같은 폴더에 있습니다. 이전 버전: `docs/guide/`, `docs/역순설계_쉽게_이해하기.md`